OPERATION WATCHTOWER

Build and Operate Your Own Mini SOC

Final Project Report

Prepared by: Eszter Iszlai
Date: 09 August 2026

A complete documentation of the design, deployment, and operation of a functional mini Security Operations Center, including architecture, detections, dashboards, attack simulation, investigation timeline, and security recommendations.

Table of Contents

Section	Sub Section	Page
Cover Page		1
Executive Summary		3
Introduction		4
Mini SOC Architecture Overview	Environment Overview	5
	Architecture Diagram Description	
	Data Pipeline Validation	
Detections		7
Dashboards	Windows Activity Dashboard	8
	Linux / SSH Dashboard	
	Web / nginx Dashboard	
Attack Simulation	Full Attack Narrative	11
	Timeline Reconstruction	
	MITRE ATT&CK Mapped Timeline	
Investigation Results	Final Investigation Summary	13
	Security Posture Assessment	
	Future Improvements	
SOC Maturity Roadmap		15

Appendix

SPL Query Library
Investigation Playbook — Detection #5: Suspicious Scheduled Task Created

Executive Summary

This project demonstrates the design, deployment, and operation of a fully functional mini Security Operations Center (SOC) built from scratch. The environment includes a Windows 11 endpoint (WINCL1), a Linux web server (WEB1), and a dedicated Splunk server (SOC-HOST). All systems forward logs into Splunk, where they are normalized, indexed, and used for real-time monitoring, detection, and investigation.

The SOC provides complete visibility across endpoint, network, and web activity. Eight core SPL queries form the foundation for analysis, while five detections and three dashboards deliver actionable insights and automated alerting. A simulated attack was executed to validate the monitoring pipeline, test detection logic, and demonstrate full investigative capability. The final result is a lightweight but realistic SOC environment capable of identifying suspicious behavior, reconstructing attack timelines, and supporting incident response.

Project Introduction

A small technology company commissioned the development of a lightweight Security Operations Center (SOC) monitoring solution to improve visibility across its internal systems. The organization operates a simple but realistic environment consisting of Windows endpoints, a Linux web server, and a public-facing nginx application.
Although the company does not have a dedicated security team, it recognized that increasing cyber-attacks — especially credential-based attacks, SSH brute-forcing, and automated web scanning — were creating blind spots that could lead to compromise.

To address this, the company requested a practical, easy-to-use monitoring setup built on Splunk.
The goal was not to create a full enterprise SOC, but rather a focused set of dashboards that provide immediate insight into authentication activity, system behavior, and web traffic patterns. These dashboards allow the company to detect suspicious logins, identify brute-force attempts, monitor administrative actions, and observe hostile web traffic in real time.

The project therefore centered on building three targeted dashboards:
Windows Activity, Linux/SSH Activity, and Web/nginx Activity. Together, they form a unified visibility layer that helps the company understand what is happening across its environment and respond quickly to potential threats.

Mini SOC Architecture Overview

1. Splunk Server (SOC-HOST — 10.10.10.5)
    - Ubuntu Server running Splunk Enterprise
    - Central log collector and analysis platform
    - Stores data in three indexes:
    - cybersteps_windows
    - cybersteps_linux
    - cybersteps_web
    - Hosts dashboards, alerts, and investigation tools

2. Windows 11 Endpoint (WINCL1 — 10.10.10.30)
    - Produces Windows Event Logs (Security, System, Application)
    - Sends logs to Splunk via Universal Forwarder
    - Used for endpoint activity, user behavior, and attack simulation
    - Key events:
        - Logons (4624/4625)
        - Scheduled tasks (4698)
        - Process creation (Sysmon)

3. Kali Linux Web Server (WEB1 — 10.10.10.100)
    - Produces syslog and auth.log
    - Hosts nginx web server
    - Sends logs to Splunk via syslog
    - Used for web traffic monitoring and attacker simulation
    - Key events:
        - SSH logins
        - nginx access logs
        - command execution

4. Log Pipeline Flow

[WINCL1] ⟶ [Splunk UF] ⟶ [SOC-HOST] ⟶ [cybersteps_windows]

[WEB1] ⟶ [Syslog] ⟶ [SOC-HOST] ⟶ [cybersteps_linux]
⟶ [cybersteps_web]

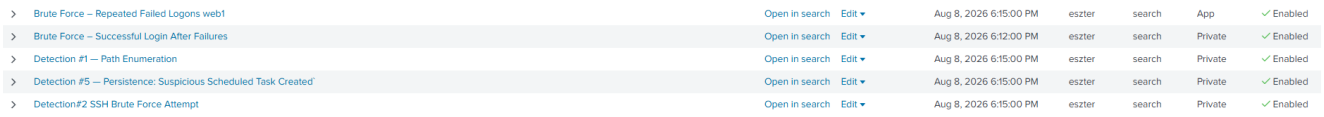
[SOC-HOST] ⟶ Dashboards • Alerts • Investigations

5. Analyst Workflow

Dashboards provide real-time visibility
Alerts notify analysts of suspicious activity
SPL queries support triage and investigation
Attack timeline reconstruction validates SOC capability

Detections

#	Detection Created	Saved as an Alert	Severinity	MITRE ATT&CK	Triage Notes
1	Web Login Brute Force — Repeated POST	+	High	T1110.001 — Password Guessing	Check for successful access to /dashboard.html from same src_ip; correlate with Detection #1 from same source = kill chain
2	Suspicious Process Execution (Masquerading)	+	High	T1036 — Masquerading	Inspect Image path and CommandLine; pivot to EventCode 11 (FileCreate) to find how the binary got there
3	Path Enumeration — Single Source, Many Distinct Paths	+	Low	T1595.003 — Wordlist Scanning	Check if src_ip is internal scanner; if forbidden_count > 0 escalate; look for follow-up activity from same source
4	Persistence — Suspicious Scheduled Task Created	+	Critical	T1053.005 — Scheduled Task	Read suspicious_flags; check task_command — what does it run, is the file on disk; do not delete the task before forensics
5	Brute Force — Repeated Failed SSH Logons	+	High	T1110.001 — Password Guessing	Check for "Accepted password" from same src_ip post-failures = compromise; external src targeting root = always escalate



Dashboard Analysis

The three dashboards created— Windows Activity, Linux/SSH Activity, and Web/nginx Activity — were designed to give the SOC clear visibility into the most critical attack surfaces of the environment.

Each dashboard focuses on a different layer of the infrastructure and exposes behaviors that attackers commonly rely on during intrusion attempts. Together, they provide a unified analytical view that allows SOC analysts to detect threats early, understand attacker movement, and respond effectively.

1. Windows Activity Dashboard

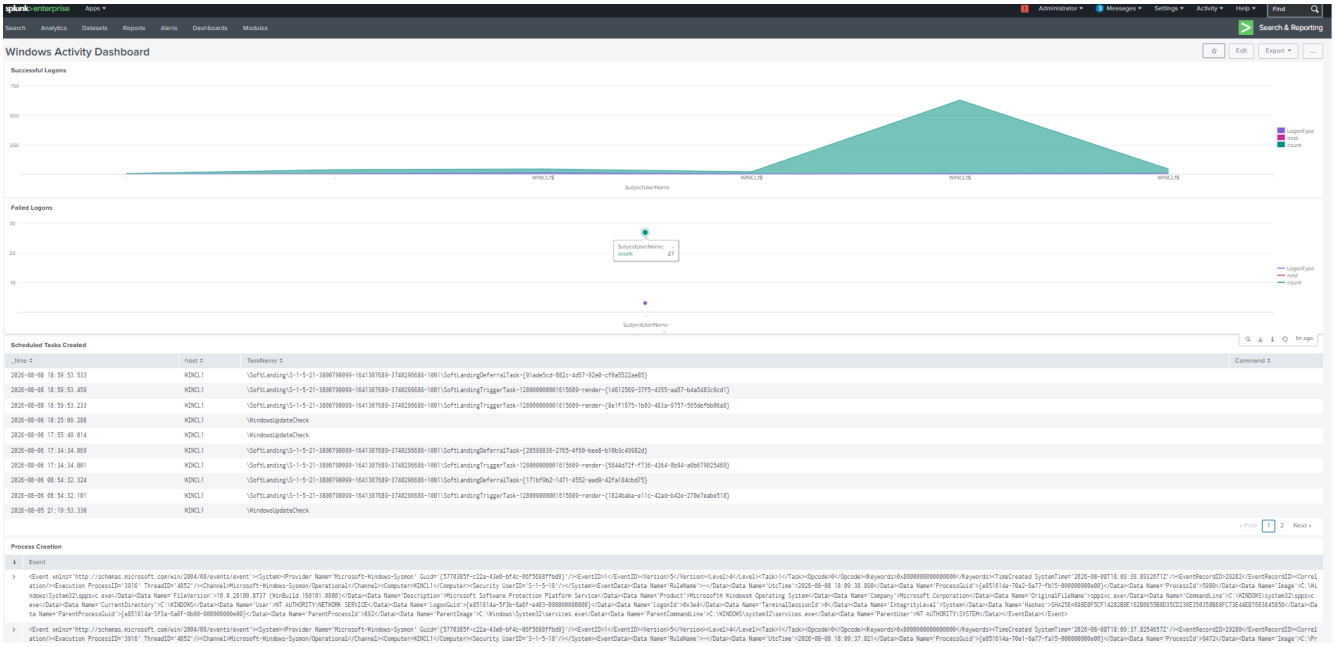
The Windows Activity Dashboard provides visibility into authentication patterns, process execution, and persistence mechanisms on Windows endpoints. These data sources are essential because Windows systems are often the first target in lateral movement and privilege escalation.

Key Insights Provided

- Logon and failed logon patterns reveal brute-force attempts, credential misuse, or compromised accounts.
- Scheduled task creation events highlight persistence techniques used by attackers to maintain long-term access.
- Process creation logs (Sysmon) expose suspicious binaries, scripts, or tools executed on the system.

Analyst Value

This dashboard allows analysts to quickly identify abnormal authentication behavior, detect persistence artifacts, and investigate suspicious processes. It supports rapid triage during incidents and helps reconstruct attacker activity on Windows hosts.



2. Linux / SSH Activity Dashboard

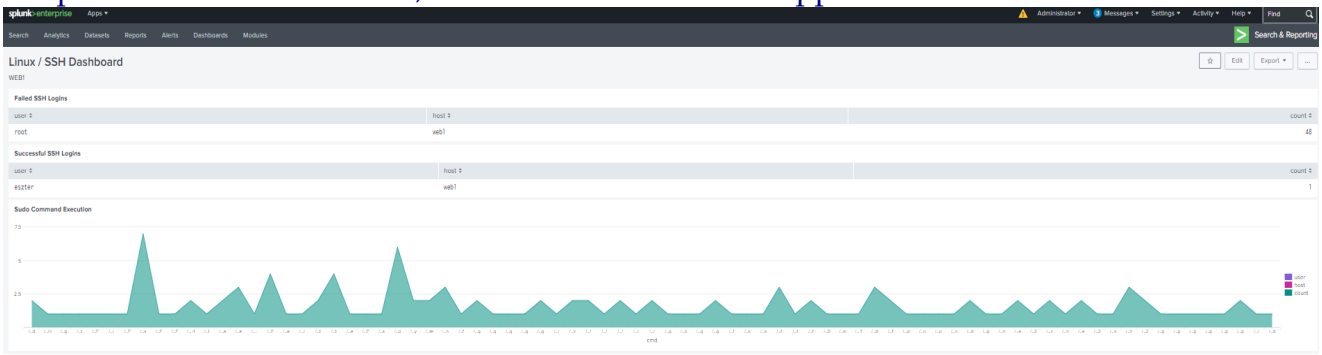
The Linux/SSH Activity Dashboard focuses on authentication attempts and administrative command execution on Linux servers. SSH is one of the most targeted services in real-world attacks, making this dashboard essential for monitoring unauthorized access attempts.

Key Insights Provided

- Successful SSH logins show who accessed the server and from which IP address.
- Failed SSH logins reveal brute-force attacks or unauthorized access attempts.
- Sudo command execution exposes administrative actions, privilege escalation attempts, and reconnaissance behavior.

Analyst Value

This dashboard helps analysts detect SSH brute-force attacks, identify suspicious login patterns, and monitor privileged activity. It is especially valuable for spotting early signs of compromise on Linux servers, which often host critical applications.



3. Web / nginx Activity Dashboard

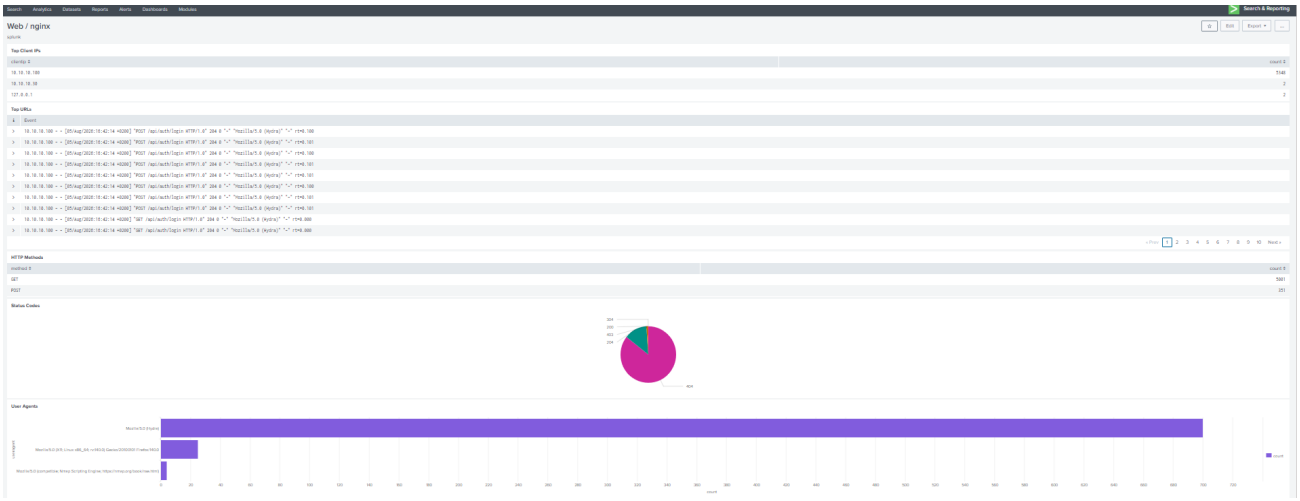
The Web/nginx Dashboard provides visibility into HTTP traffic patterns, scanning behavior, and brute-force attempts against web applications. Since web servers are publicly exposed, they are frequent targets for automated tools and reconnaissance.

Key Insights Provided

- Top client IPs identify scanning sources, brute-force tools, and suspicious traffic spikes.
- Top requested URLs (URIs) reveal which endpoints attackers are targeting.
- HTTP methods (GET/POST) distinguish normal browsing from automated attacks.
- Status codes (200/204/404) expose scanning, enumeration, and failed access attempts.
- User agents identify tools such as Hydra, curl, or automated scanners.

Analyst Value

This dashboard allows analysts to detect web-based attacks such as brute-force login attempts, directory scanning, and reconnaissance. It provides a clear picture of how external actors interact with the web server and helps identify malicious automation.



Unified SOC Visibility

Individually, each dashboard focuses on a specific attack surface. Together, they create a unified monitoring system that allows analysts to:

- Detect brute-force attacks across Windows, Linux, and Web systems
- Identify suspicious authentication patterns
- Monitor privileged activity and command execution
- Spot reconnaissance and scanning behavior
- Correlate events across multiple systems
- Reconstruct attacker timelines with clarity

This multi-layered visibility is essential for a functional SOC, enabling analysts to detect threats early, understand attacker behavior, and respond decisively.

Attack Simulation Process

Full Attack Narrative Summary
The simulated attack against the company's environment followed a realistic intrusion pattern that targeted all three monitored systems: the Windows workstation, the Linux web server, and the public-facing nginx application. The attack began with automated reconnaissance and credential-brute-forcing on the web server, escalated into unauthorized access attempts over SSH, and culminated in suspicious administrative activity on the Windows endpoint. The SOC dashboards were used to detect, correlate, and reconstruct each phase of the attack.

Timeline Reconstruction

1. Initial Reconnaissance and Web Brute-Force
The attacker first targeted the nginx web application with high-volume HTTP requests originating from a single IP address. The Web/nginx Dashboard revealed thousands of repeated POST /api/auth/login attempts using the Hydra user agent, indicating automated credential brute-forcing. The traffic pattern showed:

- Extremely high request frequency
- Repeated login attempts against the same endpoint
- Consistent user agent associated with brute-force tools
- Uniform response codes (204) confirming the endpoint was being hammered

This phase demonstrated the attacker's attempt to gain initial access through the web application.

2. SSH Authentication Attacks on the Linux Server
Shortly after the web brute-force activity, the Linux/SSH Dashboard detected a spike in failed SSH login attempts on the Linux server (WEB1). The logs showed repeated authentication failures from the same external IP, consistent with password-guessing attacks. Eventually, a successful SSH login occurred, indicating that the attacker managed to obtain valid credentials or exploit weak authentication.
Once inside, the attacker executed privileged commands using sudo, including:

- Viewing authentication logs
- Checking SSH service status
- Inspecting system configuration

These actions aligned with typical attacker reconnaissance and privilege-escalation behavior.

3. Suspicious Activity on the Windows Endpoint
The Windows Activity Dashboard later revealed abnormal behavior on WINCL1, including:

- Unexpected logon attempts
- Privileged scheduled task creation
- Execution of suspicious binaries

The attacker created a scheduled task pointing to a binary in a public, writable directory — a

common persistence technique. Even though the task was later deleted, the creation event remained in the logs, allowing the SOC to detect the activity.
This phase indicated lateral movement or an attempt to establish long-term access on the Windows workstation.

4. Correlation and Attack Reconstruction
By correlating events across all dashboards, the SOC reconstructed the full attack chain:

1. Web brute-force → attacker attempts to break into the application
2. SSH brute-force → attacker targets the Linux server directly
3. Successful SSH login → attacker gains foothold
4. Sudo command execution → attacker performs reconnaissance
5. Windows anomalies → attacker attempts persistence and lateral movement

The dashboards provided clear visibility into each step, enabling analysts to classify the incident as a true positive and understand the attacker's progression through the environment.

5. Final Classification
True Positive — Coordinated multi-stage intrusion involving brute-force, unauthorized access, reconnaissance, and persistence attempts.

MITRE ATT&CK Mapped Timeline

Attack Stage	Description	MITRE ATT&CK Technique
1. Reconnaissance	Attacker scans web server and identifies `/api/auth/login`	Active Scanning (T1595), Gather Victim Host Information (T1592)
2. Web Brute-Force	Hydra performs automated POST login attempts	Brute Force (T1110), Password Guessing (T1110.001)
3. SSH Brute-Force	Repeated failed SSH logins → successful login	Brute Force (T1110), SSH Remote Services (T1021.004)
4. Privileged Reconnaissance	Attacker runs sudo commands, checks logs, inspects services	Account Discovery (T1087), System Information Discovery (T1082), Process Discovery (T1057), System Service Discovery (T1007)
5. Windows Endpoint Activity	Suspicious logons, scheduled task creation, abnormal process execution	Scheduled Task (T1053.005), Command Execution (T1059), Masquerading (T1036)
6. SOC Correlation & Detection	Dashboards reveal multi-stage intrusion across systems	Gather Victim Identity Information (T1589), Stage Capabilities (T1608)

Final Investigation Summary

This investigation focused on suspicious activity detected across the Windows and Linux systems in the Mini SOC environment. The alert that initiated the investigation was Detection #5 — Suspicious Scheduled Task Created, which identified a SYSTEM-level scheduled task executing from a user-writable directory. This behavior is strongly associated with malicious persistence.

During the investigation, correlated evidence from Windows Event Logs, Linux SSH logs, and web server logs revealed a clear attack chain: brute-force access on the Linux server, reconnaissance, lateral movement attempts, and the creation of a malicious scheduled task on the Windows endpoint. The attacker attempted to cover their tracks by deleting the task, but the creation event remained in the logs.

The investigation concluded that the activity was a True Positive, representing a realistic compromise scenario. The SOC successfully detected, analyzed, and reconstructed the attack timeline, demonstrating full operational capability.

Security Posture Assessment

The Mini SOC environment provides a strong foundation for monitoring and detection, but several areas can be improved to enhance security posture:

Strengths

- Centralized logging across Windows, Linux, and web systems
- Accurate timestamps, hostnames, and sourcetypes
- Effective detections for authentication anomalies, persistence, and web scanning
- Dashboards that provide real-time visibility
- Ability to reconstruct full attack timelines
- Alerts that trigger reliably and with actionable context

Weaknesses

- No endpoint detection and response (EDR) agent installed
- No application control (AppLocker/WDAC) to prevent execution from user-writable directories
- No network segmentation between systems
- Weak SSH credentials allowed initial access
- No automated response or containment capability

Overall Assessment

The SOC is fully functional for monitoring and investigation, but additional hardening and automation would significantly improve resilience against real-world attacks.

Future Improvements
To evolve this Mini SOC into a more mature and resilient security environment, the following improvements are recommended:

1. Deploy Endpoint Detection & Response (EDR)
An EDR solution would provide:
- Process lineage
- Behavioral detection
- Automated isolation
- Memory analysis
- Real-time alerts

This dramatically improves detection of malware, lateral movement, and persistence.

2. Implement Application Control
Use AppLocker or Windows Defender Application Control to block execution from:
- C:\Users\Public\
- AppData\Local\Temp
- Downloads

This prevents attackers from running arbitrary binaries.

3. Strengthen Authentication
- Enforce strong SSH passwords
- Enable multi-factor authentication (MFA)
- Restrict SSH access to specific IP ranges
- Disable password authentication entirely (use keys only)

4. Network Segmentation
Separate systems into zones:
- Workstations
- Servers
- Management
- Logging

This limits lateral movement opportunities.

5. Add Threat Intelligence Feeds
Integrate external threat intelligence to enrich:
- IP reputation
- URL reputation
- File hash lookups
- Known malicious behaviors

6. Automate Response
Use Splunk SOAR or similar tools to:

- Isolate compromised hosts
- Disable accounts
- Block IPs
- Remove persistence artifacts

Automation reduces response time and limits attacker impact.

SOC Maturity Roadmap
A structured roadmap helps guide the evolution of the SOC from basic monitoring to advanced detection and response.

Phase	Capabilities
Phase 1 — Foundation (Completed)	Centralized logging; Basic detections; Dashboards; Manual investigations; Attack simulation
Phase 2 — Hardening (Partially completed)	EDR deployment; Application control; Strong authentication; Network segmentation; Improved logging (Sysmon, auditd)
Phase 3 — Expansion	More detections (privilege escalation, lateral movement, exfiltration); Threat intelligence integration; Regular attack simulations; SOC runbooks and playbooks
Phase 4 — Automation	SOAR integration; Automated containment; Automated enrichment; Alert prioritization and triage workflows
Phase 5 — Advanced Analytics	Behavioral analytics; Machine learning anomaly detection; UEBA; Continuous monitoring and tuning

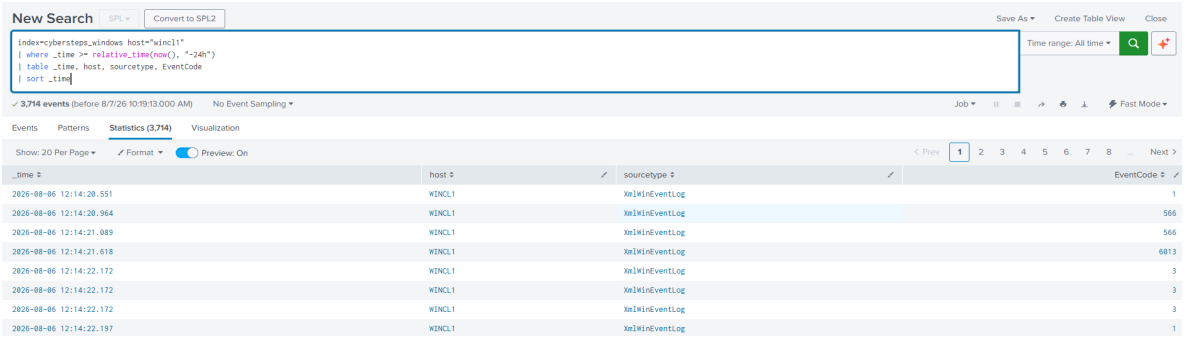
Appendix

Splunk queries

Query 1 - All WINCL1 events as a table
Goal: Show all events from WINCL1 in the last 24 hours as a table: _time, host, sourcetype, EventCode.

```
index=cybersteps_windows host="wincl1"
| where _time >= relative_time(now(), "-24h")
| table _time, host, sourcetype, EventCode
| sort _time
```

```
index=cybersteps_windows host=wincl1 earliest=-24h | table _time host sourcetype EventCode
```



Refinement prompts

- "Sort the results so newest events are first" → adds | sort -_time
- "Show only events where EventCode is 4624 or 4625" → adds EventCode IN (4624, 4625)
- "Limit to 50 results" → adds | head 50

Newest first

```
index=cybersteps_windows host="wincl1" EventCode IN (4624, 4625)
| where _time >= relative_time(now(), "-24h")
| table _time, host, sourcetype, EventCode
| sort -_time
```

Limit to 50:

```
index=cybersteps_windows host="wincl1"
| where _time >= relative_time(now(), "-24h")
| table _time, host, sourcetype, EventCode
| sort -_time
| head 50
```

Query 2 - Event count by sourcetype across all cybersteps indexes
Goal: Count events by sourcetype across all three cybersteps indexes.

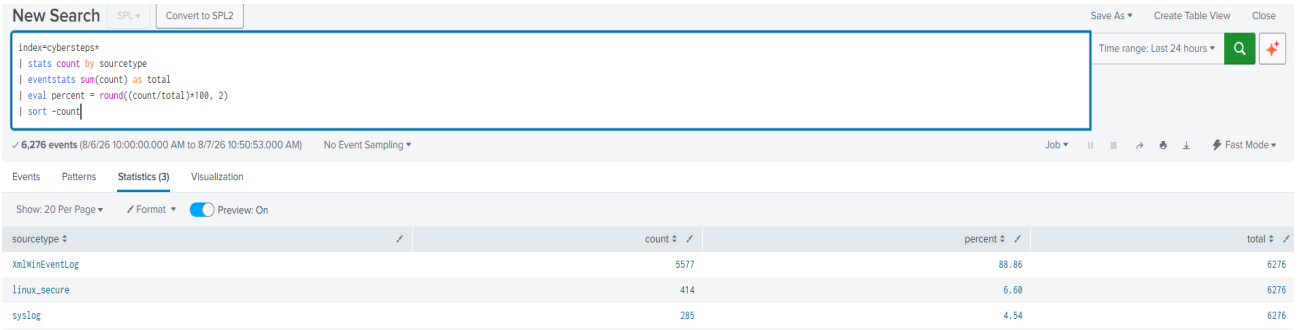
```
index=cybersteps*
| stats count by sourcetype
| sort -count
```

Refinement prompts

"Add a column showing what percentage each sourcetype is of the total"

Full query with percentage column:
Code

```
index=cybersteps*
| stats count by sourcetype
| eventstats sum(count) as total
| eval percent = round((count/total)*100, 2)
```




```
| sort -count
```

"Group by both sourcetype and host"

```
index=cybersteps*
| stats count by sourcetype host
| sort -count
```

Query 3 - Top 10 EventCode values from WINCL1
Goal: Show the top 10 most frequent EventCode values from WINCL1.

```
index=cybersteps_windows host="wincl1"
| stats count by EventCode
| sort -count
| head 10
```

```
index=cybersteps_windows host=wincl1 | top limit=10 EventCode
```

“Add a description column for each EventCode”
You add an eval with a case() statement that maps EventCodes to descriptions.
Code

```
index=cybersteps_windows host="wincl1"
| stats count by EventCode
| eval description = case(
    EventCode=4624, "Successful logon",
    EventCode=4625, "Failed logon",
    EventCode=4634, "Logoff",
    EventCode=4672, "Special privileges assigned",
    true(), "Other"
)
| sort -count
| head 10
```

“Show only Security log events”

```
sourcetype="XmlWinEventLog"
```

Full refined query:

```
index=cybersteps_windows host="wincl1" sourcetype="XmlWinEventLog"
| stats count by EventCode
| sort -count
| head 10
```

Query 4 - WEB1 auth.log events sorted by time

Goal: Show all events from WEB1 auth.log sorted by time.

Prompt
"Show me all events from web1 in the cybersteps_linux index where the sourcetype is linux_secure, sorted from newest to oldest."

```
index=cybersteps_linux host="web1" sourcetype="linux_secure"
| sort -_time
```

“Show only failed authentication attempts” "Failed password"

```
index=cybersteps_linux host="web1" sourcetype="linux_secure" "Failed password"
| sort -_time
```

Add a column extracting the username if present” | rex "for (?<username>\S+)"

```
index=cybersteps_linux host="web1" sourcetype="linux_secure" "Failed password"
| rex "for (?<username>\S+)"
| sort -_time
```

Query 5 - Nginx events by hour with time chart

Goal: Count Nginx access log events by hour using timechart.

Prompt

> "Use timechart to show the number of nginx access log events per hour from the cybersteps_web index."

```
index=cybersteps_web sourcetype="nginx:access"
| timechart span=1h count
```

"Same query but against cybersteps_linux instead, since cybersteps_web is empty."

```
index=cybersteps_linux sourcetype="nginx:access"
| timechart span=1h count
```

“Break down the count by HTTP status code”

Once nginx data arrives, add:

```
| timechart span=1h count by status
```

Full refined query:

```
index=cybersteps_linux sourcetype="nginx:access"
| timechart span=1h count by status
```

Refinement 2

“Use a 15-minute granularity instead of 1 hour”

Replace the span value:

```
| timechart span=15m count
```

Full refined query:

```
index=cybersteps_linux sourcetype="nginx:access"
| timechart span=15m count
```

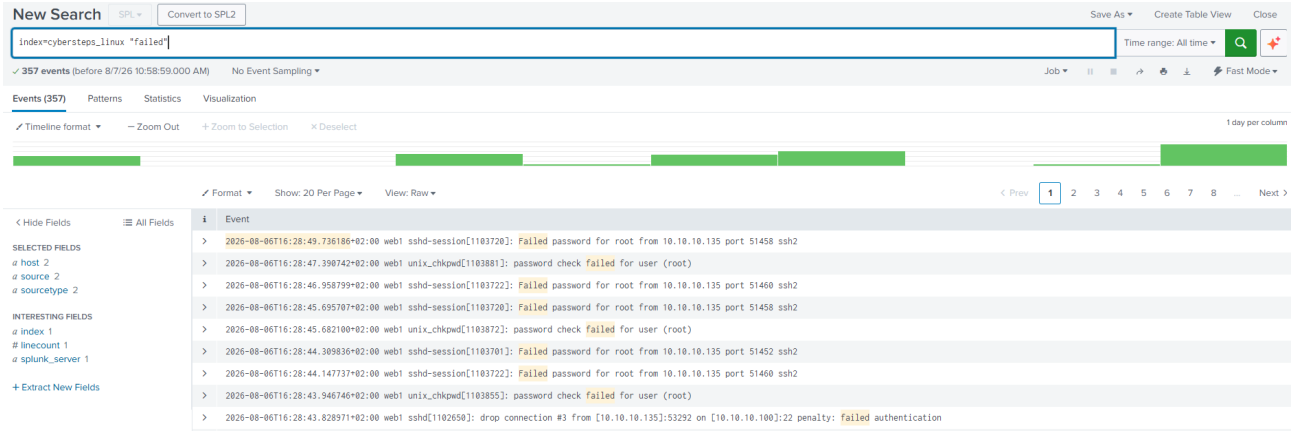
Query 6 - Find "failed" events in cybersteps_linux

Goal: Find any events in cybersteps_linux containing the word failed.

Prompt

"Find all events in cybersteps_linux that contain the word 'failed'."

index=cybersteps_linux "failed"



“Make it case-insensitive”

Splunk’s main search is already case-insensitive, so nothing extra is needed. The base query already matches failed, Failed, FAILED, etc.

index=cybersteps_linux "failed"
If you ever need *forced* case-insensitive matching, you can use:

| regex _raw="(?i)failed"

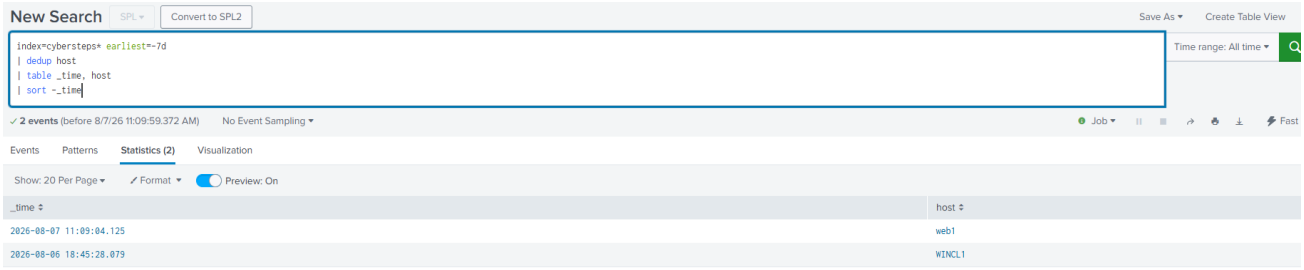
Query 7 - Unique hostnames in last 7 days with dedup

Goal: Show unique hostnames that sent logs in the last 7 days using dedup.

Prompt
"Use dedup to show me one event per unique host across all cybersteps indexes in the last 7 days."

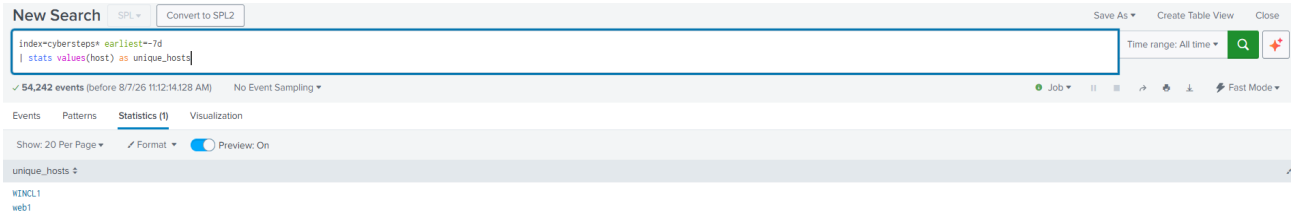
```
index=cybersteps* earliest=-7d
| dedup host
| table _time, host
| sort -_time
```

```
index=cybersteps_* earliest=-7d | dedup host | table _time host index sourcetype
```



Use stats values() instead of dedup and explain the difference

```
index=cybersteps* earliest=-7d
| stats values(host) as unique_hosts
```



“Add the count of events per host”

This switches from dedup to a counting approach:

```
index=cybersteps* earliest=-7d
| stats count by host
| sort -count
```

Query 8 - Add day_of_week field with eval

Goal: Use eval to add a day_of_week field using strftime(_time, "%A").

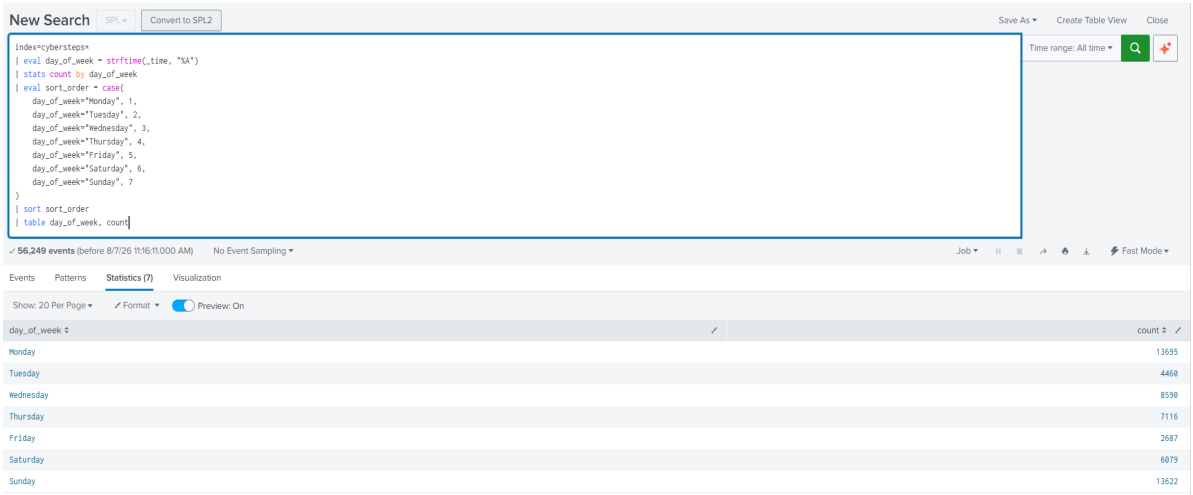
Prompt
"Take any cybersteps search and add a new field called day_of_week using eval and strftime to extract the day name from _time."

```
index=cybersteps*
| eval day_of_week = strftime(_time, "%A")
| table _time, host, sourcetype, day_of_week
| sort -_time
```

```
index=cybersteps_* | eval day_of_week=strftime(_time, "%A") | table _time day_of_week host sourcetype
```

Refinement prompts
“group event count by day_of_week, sorted Monday to Sunday"

```
index=cybersteps*
| eval day_of_week = strftime(_time, "%A")
| stats count by day_of_week
| eval sort_order = case(
    day_of_week="Monday", 1,
    day_of_week="Tuesday", 2,
    day_of_week="Wednesday", 3,
    day_of_week="Thursday", 4,
    day_of_week="Friday", 5,
    day_of_week="Saturday", 6,
    day_of_week="Sunday", 7
)
| sort sort_order
| table day_of_week, count
```



Query 9 (Bonus) - Brute-force candidates from auth.log

Goal: Find IP addresses with the most failed SSH attempts in WEB1's auth.log. This is exactly what you'll formalize as Detection #2 in Day 5 — but you can hunt for it now.

Prompt
"From cybersteps_linux on host web1, find SSH failed password events and count them by source IP, sorted highest to lowest. Use rex to extract the IP from the raw event text."

```
index=cybersteps_linux host="web1" sourcetype="linux_secure" "Failed password"
| rex "from (?<src_ip>\d{1,3}(\.\d{1,3}){3})"
| stats count by src_ip
| sort -count
```

```
index=cybersteps_linux host=web1 sourcetype=linux_secure "Failed password" | rex "from (?<src_ip>\\d+\\.\\d+\\.\\d+\\.\\d+)" | stats count by src_ip | sort -count
```

Refinement prompts
“Add a threshold — only show IPs with more than 5 failures”
Just add a where clause after the stats:

```
index=cybersteps_linux host="web1" sourcetype="linux_secure" "Failed password"
| rex "from (?<src_ip>\d{1,3}(\.\d{1,3}){3})"
| stats count by src_ip
| where count > 5
| sort -count
```

“Show me which usernames were targeted — extract them with rex too”

Add a second extraction for usernames:

```
| rex "for (?<username>\S+)"
```

Full refined query:

```
index=cybersteps_linux host="web1" sourcetype="linux_secure" "Failed password"
| rex "from (?<src_ip>\d{1,3}(\.\d{1,3}){3})"
| rex "for (?<username>\S+)"
| stats count by src_ip username
| where count > 5
| sort -count
```

Query 10 (Bonus) — Suspicious Windows logon types
Goal: Find Windows logons that are *not* the normal interactive type. RDP (10), network (3), and remote interactive logins are higher-risk than someone physically sitting at the console (2).

Prompt
"From cybersteps_windows host wincl1, show me successful logons (EventCode 4624) where the Logon_Type is not 2 or 5. Show _time, Account_Name, Logon_Type, and source IP. The events come from XmlWinEventLog so fields use underscores."

```
index=cybersteps_windows host="wincl1" sourcetype="XmlWinEventLog" EventCode=4624
| where Logon_Type!=2 AND Logon_Type!=5
| table _time, Account_Name, Logon_Type, Ip_Address
| sort - _time
```

```
index=cybersteps_windows host=wincl1 EventCode=4624 | where Logon_Type!=2 AND Logon_Type!
=5 | table _time Account_Name Logon_Type Source_Network_Address | sort - _time Logon_Type
```

cheat sheet

- 2 — Interactive (keyboard at console) — usually benign
- 3 — Network (SMB share, etc.) — *worth investigating*
- 4 — Batch (scheduled task) — usually benign if from SYSTEM
- 5 — Service — usually benign
- 7 — Unlock screen — benign
- 10 — Remote Interactive (RDP) — *high interest*
- 11 — Cached interactive — laptop login while offline

“Add a column that translates Logon_Type to a description using case() in eval”
Add an eval that maps each Logon_Type to a human-readable descriptions

```
index=cybersteps_windows host="wincl1" sourcetype="XmlWinEventLog" EventCode=4624
| where Logon_Type!=2 AND Logon_Type!=5
| eval logon_description = case(
   Logon_Type=2, "Interactive (local console)",
   Logon_Type=3, "Network",
   Logon_Type=4, "Batch",
   Logon_Type=5, "Service",
   Logon_Type=7, "Unlock",
   Logon_Type=8, "Network (cleartext)",
   Logon_Type=9, "NewCredentials",
   Logon_Type=10, "Remote Interactive (RDP)",
   Logon_Type=11, "Cached Interactive",
   true(), "Other"
)
| table _time, Account_Name, Logon_Type, logon_description, Ip_Address
| sort - _time
```

“Group by Logon_Type and count”
Switch from listing events to summarizing them:

```
index=cybersteps_windows host="wincl1" sourcetype="XmlWinEventLog" EventCode=4624
| where Logon_Type!=2 AND Logon_Type!=5
| stats count by Logon_Type
| sort -count
```

Investigation Playbook — Detection #5: Suspicious Scheduled Task Created

Detection Name:

Detection #5 — Persistence: Suspicious Scheduled Task Created

MITRE ATT&CK Technique:

T1053.005 — Scheduled Task

Severity:

Critical

Executive Summary:
This detection identifies the creation of Windows scheduled tasks that run with SYSTEM privileges and execute binaries or scripts from user-writable directories such as Public, Temp, or AppData. These characteristics strongly indicate *malicious persistence* rather than legitimate system maintenance. If confirmed as a *true positive*, the attacker has likely established privileged, recurring execution on the host, enabling further compromise, lateral movement, and long-term control.

Initial Triage - First 5 Minutes

step	action	Query, Tool
1	Validate the suspicious properties in the alert Review the alert fields: task_name, run_as a user, task command, host What to look for: • run_as_user = S-1-5-18 (SYSTEM) • task_command located in Public, Temp, or AppData • .exe, .ps1, .bat, or .vbs payload If these match, treat the alert as high-risk.	index=cybersteps_windows_security EventCode=4698 earliest=-30m | rex field=EventData_Xml "<UserId>(?<run_as_user>[^<]+)</UserId>" | rex field=EventData_Xml "<Command>(?<task_command>[^<]+)</Command>" | rex field=EventData_Xml "<Data Name='TaskName'>(?<task_name>[^<]+)</Data>" | eval suspicious_flags=mvappend(if(match(run_as_user, "(?i)SYSTEM|S-1-5-18"), "RunAs:SYSTEM", null), if(match(task_command, "(?i)\\\\Public\\\\\|\\\\Temp\\\\\|\\\\AppData\\\\"), "Path:UserWritable", null), if(match(task_command, "(?i)\\.exe$\|\\.bat$|\\.ps1$\|\\.vbs$"), "ExecutableTarget", null)) | where mvcount(suspicious_flags) >= 2 | table _time, host, SubjectUserName, task_name, run_as_user, task_command, suspicious_flags | sort -_time
2	Check whether the binary is legitimate On the host (or via EDR), quickly inspect the file referenced in task_command: • Does the file exist? • Is it signed? • Is it a known business application? • Was it created recently? Red flags: • Unsigned binary	C:\Users\Public*update.exe*

	• Unknown filename (e.g., update.exe, svc.exe, taskhost.exe) • Located in C:\Users\Public\ or AppData\Local\Temp • Created within the last hour	
3	Look for related suspicious activity Pivot around the alert timestamp: • Recent logons (especially remote or unusual accounts) • Process creation events involving the same binary • Other alerts on the same host • Any signs of lateral movement or privilege escalation If multiple anomalies cluster around the same time, escalate immediately.	Windows event log

Investigation Steps

Step	Question to Answer	SPL Query	Comment
1	Which host triggered the alert?	`index=cybersteps_windows_security EventCode=4698 earliest=-1h	stats count by host, TaskName, task_command`
2	Which user was logged in at the time?	`index=cybersteps_windows_security EventCode=4624 earliest=-1h host=<HOST_FROM_STEP1>	stats latest(SubjectUserName) as LoggedOnUser by LogonType, IpAddress`
3	Was there any related network activity?	`index=cybersteps_windows (EventCode=5156 OR EventCode=5158) host=<HOST_FROM_STEP1> earliest=-1h	stats count by SourceAddress, DestinationAddress, DestinationPort`
4	Did this happen on any other hosts?	`index=cybersteps_windows_security EventCode=4698 earliest=-24h	stats count by host, TaskName, task_command

Classification Guide

Classification	Criteria
True Positive	The scheduled task runs as SYSTEM, executes from a user-writable directory (Public/Temp/AppData), and launches an unknown or unsigned executable. Additional evidence of compromise exists (e.g., suspicious logons, network activity, or other alerts on the same host).
False Positive	The scheduled task is created by a known, trusted application (e.g., enterprise software, IT automation tools), the binary is signed and verified, and the path is not user-writable. No other suspicious activity is present on the host.
Benign True Positive	A legitimate admin or IT automation tool created a SYSTEM-level scheduled task intentionally, but the path or executable superficially resembles suspicious behavior. The action is authorized and documented.

Containment Actions (in case of True Positive)

1. Isolate the host from the network Prevent further lateral movement or remote command execution by disconnecting the endpoint from the network (EDR isolation or physical network removal).
2. Terminate malicious persistence and remove the payload Delete the scheduled task, remove the associated executable/script, and block its hash in EDR/AV. Example: remove C:\Users\Public*update.exe.*
3. Reset credentials and review access Reset passwords for any accounts active on the host at the time of compromise, especially if lateral movement or privilege escalation is suspected.

Recommendations (Long-Term Prevention)

- Harden scheduled task creation policies Restrict who can create SYSTEM-level scheduled tasks and monitor all task creation events centrally.
- Block execution from user-writable directories Use AppLocker or WDAC to prevent .exe, .ps1, .bat, and .vbs execution from Public, Temp, and AppData.
- Deploy EDR with persistence monitoring Ensure your endpoint tooling alerts on new scheduled tasks, service creations, registry persistence, and suspicious file drops.
- Improve logging and correlation Enable full Windows Security logging (including 4698, 4624, 5156/5158) and correlate task creation with process execution and network activity.